

C++ | CPL

Summary

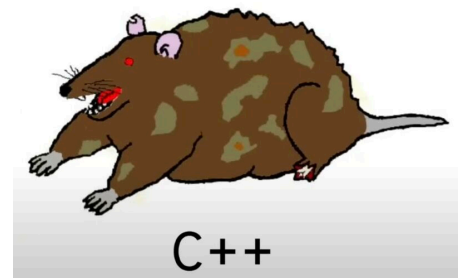
CONTENTS

1. Why C++?	3
2. Intro	3
2.1. Compilation	4
2.2. Declarations and definitions	4
2.2.1. #include guards	4
2.2.2. Functions	4
2.2.3. Values and references	4
2.2.4. Alias Declarations	5
2.3. Libraries	5
3. Values and Streams	5
3.1. Variables	5
3.2. Literal Values	6
3.3. Expressions	6
3.3.1. Type Conversion	6
3.3.2. Assignment Operation	7
3.3.3. Logic Operations	7
3.3.4. Floating Point Numbers (IEEE754)	7
3.3.5. Strings	7
3.4. Basic Streams	7
3.4.1. Errors	7
3.4.2. std::string	8
3.4.3. int	8
3.4.4. Boolean Conversion	8
3.4.5. IO Headers	9
4. Sequences and Iterators	9
4.1. std::array<T, N>	9
4.2. std::vector<T>	10
4.3. Iteration	11
4.3.1. Algorithms	11
4.3.2. Ranges	12
4.3.3. Putting it all together	12
4.4. Mutating	12
4.4.1. Filling	12
4.5. Finding and counting elements	13
4.6. I/O	13
5. Functions	14
5.1. Basics	14
5.1.1. Arguments	14
5.1.2. Return types	14
5.1.3. Overloading	15
5.1.4. Default Arguments	15

5.1.5. Functions as Parameters	15
5.1.6. Lambdas	16
5.2. Failing	17
5.2.1. Exceptions	18
Bibliography	20

1. WHY C++?

- Multi-paradigm language, now you have a thousand and one way to shoot yourself in your foot and obliterate your PC
- Being called stupid when you think that the move function actually moves things or if you confuse a const pointer with a pointer of const
- ISO standard: C++23. The higher the number the hotter the mess of poorly designed choices that have accumulated over decades into a chaotic pile of obsolescence
- Good excuse for alcoholism
- Zero-cost abstractions (if you like learning the whole stdlib by heart and don't mind the cost of debugging template errors for hours on end)
- Headaches (if you're a masochist)
-
- It's a never ending journey. Even if you dedicate your whole life to learning C++, you'll never reach the day where you'll know all of the stdlib.
- "Undefined behavior" is defined (in the standard)
- Ever wanted to have 100 different ways to do the same thing? Now you can
- Has a pretty mascot [\[1\]](#)



2. INTRO

- Implicitly returns 0
- Return types are written in front of the function name or as trailing return-types in declarations (because who needs consistency, amirite)

Nice and clean: `int main() { }` Insanity: `auto main() → int { }`

- Header files can be either `*.hpp` or `*.h`
- Regarding include order: It is good practice to include own-headers ("`"`") before system-headers ("`<`"). The first is usually used for the header files belonging to the project itself, while the latter is used for includes of the standard library and external libraries. However, ***this difference is only conventional, as any toolchain could just specify their own lookup rules for the two distinct kinds of includes.***

Do's	Don'ts
	Using global variables
	Redundant returns in <code>main()</code> that would be implicit
	<code>using namespace</code>
	Not initializing a variable upon definition using <code>{}</code>
	Not using <code>const</code> enough
	Using <code>sdt::cin</code> and <code>sdt::cout</code>
	Not checking <code>stream.good()</code>
	Not removing stream fail flag with <code>stream.clear()</code>
	Not using <code>#include</code> guards
	Using plain C-Arrays

<i>Do's</i>	<i>Don'ts</i>
	Using plain <code>for(int i; ...)</code> loops
	Returning from a function by <code>const</code> value (unnecessary and stupid)
	Returning reference to a local variable
	Throwing primitives. Don't ask why exactly should you be able to do this in the first place.
	TODO: make this column real big for the lulz

2.1. COMPILATION

3 Phases of compilation:

Preprocessor: Textual replacement of preprocessor directives (`#include`)

Compiler: Translation of C++ code into machine code (source file to object file)

Linker: Combination of object files and libraries into libraries and executables

2.2. DECLARATIONS AND DEFINITIONS

All things with a name that you use in a C++ program must be declared before you can do so

- E.g. a function that you call (major difference from C)
- A type that you use for a variable (except some built-ins)
- A variable that you use

There can only be one definition of the same function (**One Definition Rule**/ODR), in contrast to declaration, which can be done several times.

2.2.1. `#include` guards

`#include` guards ensure that a header file is only included once. Necessary for functions and class type definitions, good practice either way.

```
#ifndef GUARD_HPP_
#define GUARD_HPP_
// ...
#endif /* GUARD_HPP_ */
```

2.2.2. Functions

Declarations

```
auto <function-name>(<parameters>) → <return-type>;
<return-type> <function-name>(<parameters>);
```

Definitions

```
auto <function-name>(<parameters>) → <return-type> { /*body*/ }
<return-type> <function-name>(<parameters>) { /*body*/ }
```

If a function has a non-void return type it must return a value on every path (or throw an exception).

2.2.3. Values and references

```
struct Point {
    int x;
```

```
int y;  
};
```

Copying vs. Sharing:

```
Point point{1, 20};
```

```
Point copiedPoint{point};  
Point & sharedPoint{point};
```

```
auto copydPointParam(Point point) → Point { }  
auto referencePointParam(Point & point) → Point { }
```

2.2.4. Alias Declarations

A type alias can help to abbreviate type names.

```
using <alias-name> = <type>;
```

Example 1:

```
using input = std::istream_iterator<std::string>;  
input eof{};  
input in{std::cin};
```

2.3. LIBRARIES

- [catch2](#)
- [boost](#)

TODO:
cmake

3. VALUES AND STREAMS

3.1. VARIABLES

```
<type> <variable-name>{<initial-value>;
```

Using = or {} for initialization with a value supplied we can have the compiler determine its type.

```
auto const i = 5;
```

Initialization might be omitted but that is bad practice and potentially dangerous.

```
double x;
```

Adding the const keyword in front of the name makes the variable a single-assignment variable, aka a constant which is immutable and must be initialized. Some constants are required to be fixed at compile time (**constexpr**). The keyword **const** also appears in other contexts.

```
int const theAnswer{42};  
double constexpr pi{3.14};
```

You should use **const** whenever possible for non-member variables.

3.2. LITERAL VALUES

<i>Literal Example</i>	<i>Type</i>	<i>Value</i>
'a'	char	Letter a, value: 97
'\n'	char	<NL> character, value: 10
'\x0a'	char	<NL> character, value: 10
1	int	1
42L	long	42
5LL	long long	5
int{} // (not really a literal)	int	0 (default value)
1u	unsigned int	1
42uL	unsigned long	42
5uLL	unsigned long long	5
020	int	16 (octal 20)
0x1f	int	31 (hex 1F)
0XFULL	unsigned long long	15 (hex F)
0.f	float	0
.33	double	0.33
1e9	double	1000000000 (10 ⁹)
42.E-12L	long double	0.00000000042 (42*10 ⁻¹²)
.3L	long double	0.3
"hello"	char const [6]	Array of 6 chars: h e l l o <NUL>
"\012\n\\"	char const [4]	Array of 4 chars: <NL> <NL> <NUL>

3.3. EXPRESSIONS

	<i>Arithmetic</i>	<i>Bit-operators</i>	<i>Logic</i>
<i>Unary</i>	+ - ++ --	~	!
<i>Binary</i>	+ - * / %	& ^ << >>	&& < > ≤ ≥ = ≠
<i>Tertiary</i>			? :

- Fraction results of integer operations are always rounded down
- [Operator precedence](#)

3.3.1. Type Conversion

- Integer to boolean conversion 0 → false / every other value → true
- Automatic type conversion if values of different types are combined in an expression, **unless in braced initialization**
- Dividing integers by zero is **undefined behavior**

3.3.2. Assignment Operation

```
// vvvvv lvalue
    myVar = 6 * 7;
//      ^^^^^ rvalue
```

3.3.3. Logic Operations

Logical operators and conditional statements are generous to accept numeric values as statement of truth

```
// v returns boolean
if (a < b < c);
//      ^ coalesces to 0 if false, 1 if true
```

3.3.4. Floating Point Numbers (IEEE754)

- Use `double` – usually most efficient on current hardware and default for floating point literals.
- Use `float` only if memory consumption is utmost priority and precision and range can be traded (on 64bit often not beneficial).
- Remember there are legal double values that are not numbers: `NaN`, `+Inf`, `-Inf`.
- Comparing floating points for equality (`==`) is usually wrong
 - In Catch2 you have the option to specify a matcher that checks in a relative range to the expected value `REQUIRE_THAT(actual, Catch::Matchers::WithinRel(expected, delta))`;

3.3.5. Strings

`std::string` is C++'s type for representing sequences of char (which is often only 8 bit). For working with non-ASCII characters an external library is advised. It is mutable and iterable with iterators.

```
#include <string>
std::string statement{"Rust ftw"};
```

String literals like `"ab"` are not of type `std::string`, they're a null-terminated array of const characters (`char const[3]`). But `"ab"`'s is an `std::string` but requires `using namespace std::literals`.

3.4. BASIC STREAMS

TODO:

`std::noskipws` and other stream manipulators

Streams aren't values, because they cannot be copied. So functions taking a stream object must take it as a reference.

Pre-defined globals: `std::cin` `std::cout`, should only be used in `main()`. "shift" operators read into variables or write values and can be chained:

```
std::cin >> x;
std::cout << "the value is " << x << '\n';
```

Output can be formatted using [I/O manipulators](#).

TODO:

W2 slides 43

3.4.1. Errors

Streams have a state that denotes if I/O was successful or not. If a previous read already failed, subsequent reads fail as well.

State Bit Set	Query	Entered
<none>	is.good()	initial is.clear()
failbit	is.fail()	formatted input failed
eofbit	is.eof()	trying to read at end of input
badbit	is.bad()	unrecoverable I/O error

- Formatted input on stream is must be checked for is.fail() and is.bad()
- If failed, is.clear() the stream and consume invalid input characters before continuing

3.4.2. std::string

Reading a std::string can not go wrong, unless the stream is already !good()

- The content of the std::string is replaced
- Maybe the std::string is empty after reading

3.4.3. int

Reading an int results in:

- No error recovery
- One wrong input puts the stream into status fail
- Characters remain in input

Robust way of reading an int:

```
auto readInt(std::istream & in) → int {
    std::string line{};
    while (getline(in, line)) {
        std::istringstream is{line};
        int res{-1};
        if (is >> res) {
            return res;
        }
    }
    return -1;
}
```

TODO:

W2 slides 41

3.4.4. Boolean Conversion

Result of is >> res is the std::istream object itself. The stream object converts to bool (in if and loop conditions):

- true if the last reading operation has been successful
- false if the last reading operation failed somehow (formatting, stream end or another problem)

good() for pre-checking “can I still read?” and bool conversion for post-checking “did I read correctly?”.

ios_base::iostate flags			basic_ios accessors					
eofbit	failbit	badbit	good()	fail()	bad()	eof()	operator bool	operator !
false	false	false	true	false	false	false	true	false
false	false	true	false	true	true	false	false	true
false	true	false	false	true	false	false	false	true
false	true	true	false	true	true	false	false	true
true	false	false	false	false	false	true	true	false
true	false	true	false	true	true	true	false	true
true	true	false	false	true	false	true	false	true
true	true	true	false	true	true	true	false	true

3.4.5. IO Headers

iosfwd: contains only the declarations for `std::ostream` and `std::istream`.

In header files (`.hpp`) this is usually sufficient when the streams are only used in function declarations

istream and ostream: contain the implementation of the corresponding stream and operators.

Usually, these are required in source files (`.cpp`) when the streams are actually used in functions

iostream: contains all of the above and additionally `std::cout`, `std::cin`, `std::cerr`.

This is only required in the source file containing the `main()` function, because only there the global standard IO variables shall be used

4. SEQUENCES AND ITERATORS

4.1. `STD::ARRAY<T, N>`

```
#include <array>
```

```
std::array<int, 6> emptyArray{};
```

Is a fixed-size Container

- T is a template type parameter (= placeholder for type)
- N is a positive integer, template non-type parameter (= placeholder for a value)

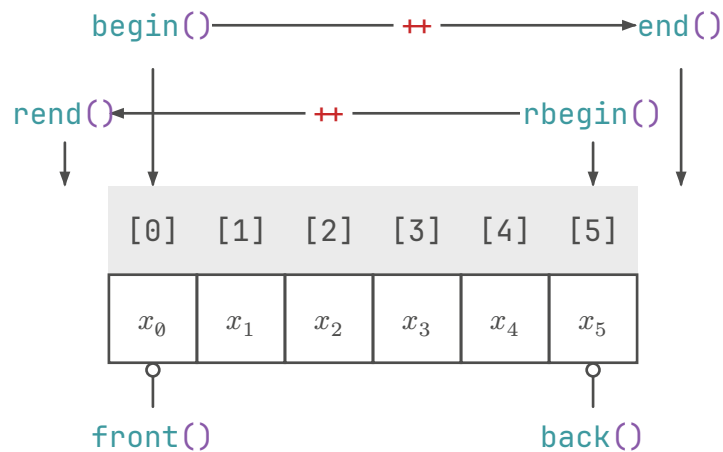
It can be initialized with a list of elements

- The size of an array must be known at compile-time and cannot be changed
- Otherwise, it contains N default-constructed elements

The size is bound to the array object and can be queried using `.size()`

Element access using subscript operator `[]` or `.at()`

- `.at()` throws `std::out_of_range` exception on invalid index access
- `[]` has undefined behavior on invalid index access



4.2. `std::vector<T>`

```
#include <vector>
```

```
std::vector<int> numbers{1, 2, 3, 4, 5};
```

Is a Container = contains its elements of type `T` (no need to allocate them). Works similar to an `java.util.ArrayList<T>` in java, although it doesn't store the Elements as references but as copies (use `std::vector<&T>` for that purpose).

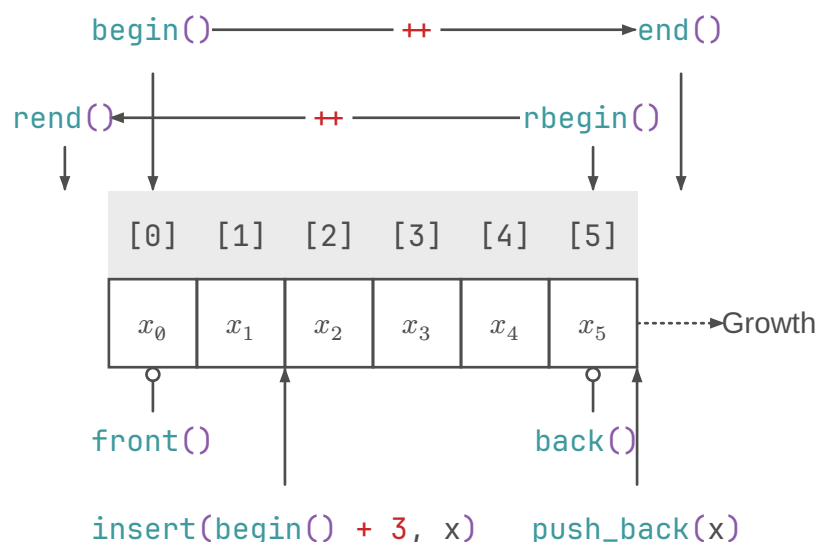
`std::vector` can be initialized with a list of elements

- The list can be empty: `std::vector<double> vd{};`
- Other construction means might need parentheses (legacy)

When an initializer is given, the element type can be deduced! `std::vector{1, 2, 3, 4, 5};`

Parenthesis at definition allow providing initial size, when type of elements is not numeric
`std::vector<std::string> words{6}`

Index variable type is "unsigned" `std::size_t` or `std::vector<T>::size_type`



4.3. ITERATION

	<i>const</i>	<i>non-const</i>
<i>reference</i>	<code>for (auto const & cref : v) { }</code>	<code>for (auto & ref : v) { }</code>
<i>copy</i>	<code>for (auto const ccopy : v) { }</code>	<code>for (auto copy : v) { }</code>

Only use this if the iterator is required in the loop:

```
for (auto it = std::begin(v); it != std::end(v); ++it) {
    std::cout << (*it)++ << ", ";
}
```

Guarantee to just have read-only access: `std::cbegin()` and `std::cend()`

4.3.1. Algorithms

```
#include <algorithm>
```

Each algorithm takes iterator arguments: The range(s) of elements to apply an algorithm to is specified by iterators.

```
auto count_blanks(std::string s) → size_t {
    return std::count(s.cbegin(), s.cend(), ' ');
}
```

Example 2: Sum

```
#include <numeric>
std::vector<int> v{5, 4, 3, 2, 1};
std::cout << std::accumulate(std::cbegin(v), std::cend(v), 0) << "=sum\n";
```

Example 3: Nr. of elements

```
#include <iterator>
void printDistanceAndLength(std::string s) {
    std::cout << "distance: " << std::distance(s.begin(), s.end()) << '\n';
    std::cout << "in a string of length: " << s.size() << '\n';
}
```

Example 4: Foreach

```
auto print(int x) → void {
    std::cout << "print: " << x << '\n';
}
auto printAll(std::vector<int> v) → void {
    std::for_each(std::cbegin(v), std::cend(v), print);
}
```

Example 5: Foreach with lambdas

```
auto printAll(std::vector<int> v, std::ostream & out) → void {
    std::for_each(std::cbegin(v), std::cend(v), [&out](auto x) {
        out << "print: " << x << '\n';
    });
}
```

4.3.2. Ranges

```
auto printAll(std::vector<int> v, std::ostream & out) → void {
    std::ranges::for_each(v, [&out](auto x) {
        out << "print: " << x << '\n';
    });
}
```

4.3.3. Putting it all together

Iterators connect containers and algorithms.

Containers	Iterators	Algorithms
std::vector<T>	std::begin()	std::count(b, e, val)
std::string	std::end()	std::ranges::count(r, val)
std::set<T>	std::rbegin()	std::find(b, e, val)
std::map<K, V>	std::rend()	std::accumulate(b, e, start)
...	...	std::copy(b, e, b_target)
		...

4.4. MUTATING

Append: `v.push_back(<value>);`

Insert anywhere: `v.insert(<iterator-position>, <value>);`

When using the `std::copy` algorithm the target has to be an iterator too

```
std::copy(<in-begin-iterator>, <in-end-iterator>, <out-begin-iterator>);
std::ranges::copy(<in-range>, <out-begin-iterator>);
```

To append elements to another list, use `std::back_inserter()`:

```
std::vector<int> source{1, 2, 3}, target{};
std::copy(source.cbegin(), source.cend(), std::back_inserter(target));
std::ranges::copy(source, std::back_inserter(target));
```

4.4.1. Filling

Filling a vector with the same values can be done in multiple ways:

```
// std::fill requires a vector with existing elements to be overwritten
std::vector<int> v{};
v.resize(10);
std::fill(std::begin(v), std::end(v), 2);
std::ranges::fill(v, 2)
```

```
// or
std::vector<int> v(10);
std::fill(std::begin(v), std::end(v), 2);
std::ranges::fill(v, 2)
// or
std::vector<int> v(10, 2);
```

Filling a vector with different values can also be done in multiple ways (wowzers):

The algorithms `std::generate()` and `std::generate_n()` fill a range with computed values. Either use `std::back_inserter` or a non-empty container respectively. The `std::iota()` algorithm fills a range with subsequent values.

4.5. FINDING AND COUNTING ELEMENTS

`std::(ranges::)find()` and `std::(ranges::)find_if()` return an iterator to the first element that matches the value or condition. If no match exists the end of the range is returned.

Similarly `std::(ranges::)count()` and `std::(ranges::)count_if()` return the number of matching elements in a range.

4.6. I/O

Streams cannot be used with algorithms directly.

Containers	Iterators	Algorithms
std::ostream std::istream	std::ostream_iterator() std::istream_iterator()	std::count(b, e, val) std::ranges::count(r, val) std::copy(b, e, b_target) ...

`std::istream_iterator<T>` reads values of type `T` from the given `std::istream`. End iterator is the default constructed `std::istream_iterator<T>{};` it ends when the stream is no longer `good()`.

`std::ranges::istream_view<T>` combines `in` and `eof`.

`std::istream_iterator<T>` uses operator `>>` for input, which skips white space. For an exact copy, use `std::istreambuf_iterator<char>`, which uses `std::istream::get()` to get every character. This only works with `char`-like types.

Example 6:

```
using input = std::istreambuf_iterator<char>;
input eof{};
input in{std::cin};
std::ostream_iterator<char> out{std::cout, " "};
std::copy(in, eof, out);
```

To fill a vector from a stream you can either use `copy` with `std::back_inserter(v)` (It uses `v.push_back()` internally)

```
using input = std::ranges::istream_view<int>;
std::vector<int> v{};
std::ranges::copy(input{std::cin}, std::back_inserter(v));
```

or construct the `std::vector<T>` directly from two iterators

```
using input = std::istream_iterator<int>;
input eof{};
std::vector<int> const v{input{std::cin}, eof};
```

5. FUNCTIONS

5.1. BASICS

5.1.1. Arguments

	<i>const</i>	<i>non-const</i>
<i>reference</i>	<code>auto f(char const & c) → void { }</code>	<code>auto f(char & c) → void { }</code>
<i>copy</i>	<code>auto f(char const c) → void { }</code>	<code>auto f(char c) → void { }</code>

5.1.2. Return types

In function definitions the trailing return-type could be omitted. The actual return type will be deduced from the return statements in the function's body.

	<i>const</i>	<i>non-const</i>
<i>reference</i>	<code>auto f() → type const & { }</code>	<code>auto f() → type & { }</code>
<i>value</i>	<code>auto f() → type const { }</code>	<code>auto f() → type { }</code>

Example 7: Incorrect

```
#include <iostream>

// here the compiler can't deduce the return type!
auto maxValue(int f, int s, int t);

// this won't compile
int main() {
    std::cout << maxValue(1, 2, 3);
}
```

5.1.2.1. Const reference

`const &` extends the life-time of the temporary object, until the end of the block.

Example 8: The POI will be copied into the returned `std::vector` object

```

auto createPOI(Coordinate) → POI;
auto allPOIs(Coordinate const location) → std::vector<POI> {
    POI const & migros = createPOI(location);
    return std::vector{migros};
}

```

5.1.3. Overloading

```

auto incr(int & var) → void;
auto incr(int & var, unsigned delta) → void;

```

The same function name can be used for different functions if parameter number or types differ

- Functions cannot be overloaded just by their return type
- If the parameter type is only different in reference/object there will be ambiguities
- If the parameter is only different in type, there might be ambiguities (e.g. passing a **long** into **int** vs. **double**)

Resolution of overloads happens at compile-time (Ad hoc polymorphism). That also means that the internal name of a function also contains its parameter types as significant information.

5.1.4. Default Arguments

```

auto incr(int & var, unsigned delta = 1) → void;

```

A function declaration can provide default arguments for its parameters from the right. Definition doesn't need to/shouldn't repeat:

```

auto incr(int & var, unsigned delta) → void {
    var += delta;
}

```

If n default arguments are provided, the behavior is, as if $n + 1$ versions of the function were declared.

5.1.5. Functions as Parameters

```

auto applyAndPrint(double x, auto f(double) → double) → void {
    std::cout << "f(" << x << ") = " << f(x) << '\n';
}

```

Type signatures (legacy):

```

auto f(double) → int
int f(double)
auto (&ref)(double) → int
int (&ref)(double)

```

Drawback: A function parameter declared in this way does not accept a lambda with a capture. Use `std::function`.

5.1.5.1. std::function

```

#include <functional>

```

Modern C++ approach: `std::function` template, which also allows passing lambdas (with capture)

```

std::function<auto(double) → int>
std::function<int(double)>

```

Example 9:

```

auto applyAndPrint(double x, std::function<auto(double) → double> f) →
void {
    std::cout << "f(" << x << ") = " << f(x) << '\n';
}
auto main() → int {
    double factor{3.0};
    auto const multiply = [factor](double value) {
        return factor * value;
    };
    applyAndPrint(1.5, multiply);
}

```

5.1.6. Lambdas

```

[<capture>](<parameters>) → <return-type> {
    <statements>
}

```

Capture names variables taken from the surrounding scope, or define new ones (= copy, & reference, rename possible, type deduced). The return type can be omitted if **void** or if inferrable for the compiler. Parameters can be **auto** if inferrable from context.

```

auto g = [](char c) → char {
    return std::toupper(c);
};
g('a');

```

Captured local copies are immutable, unless lambda is declared mutable and lives as long as the lambda lives.

```

int x = 5;
auto l = [x]() mutable {
    std::cout << ++x;
};

```

Capturing a local variable by reference requires the referenced variable to live at least as long as the lambda.

Capturing all (referenced) local variables by value:

```

int x = 5;
auto l = [=]() mutable {
    std::cout << ++x;
};

```

Capturing all (referenced) local variables by reference:

```

int x = 5;
auto const l = [&]() {
    std::cout << ++x;
};

```

Referenced variables will allow modification, unless it is originally declared `const`

Capturing **this** pointer allows accessing and modifying members of the class

```
struct S {
    auto foo() → void {
        auto square = [this] {
            member *= 2;
        };
    }
private:
    int member{};
};
```

New local variable can be specified in capture

- New variable in capture has type `auto`
- Can be modified if lambda is mutable

```
auto squares = [x = 1]() mutable {
    std::cout << (x *= 2);
};
```

In captures multiple variables can be combined and separated with commas (,)

TODO:

Within a single expression, such as a function call, sequence of evaluation is undefined! (except for the comma operator ,)

5.2. FAILING

A function can fail if the **precondition** is violated (e.g. negative index, divisor is zero, ...) or if the **postcondition** could not be satisfied (e.g. resources not available, cannot open file, ...)

A function without preconditions has a so-called **wide contract** as opposed to **narrow contract**.

What should you do, if a function cannot fulfill its purpose?

- 1) Ignore the error and provide potentially **undefined behavior**
 - Relies on the caller to satisfy all preconditions
 - Viable only if not dependent on other resources
 - Most efficient implementation (no unnecessary checks)
 - Simpler for the implementer but harder for the caller
- 2) Return a **standard result** to cover the error
 - Reliefs the caller from the need to care if it can continue with the default value
 - Can hide underlying problems
 - Often better if caller can specify its own default value
- 3) Return an **error code** or error value
 - Only feasible if result domain is smaller than return type
 - Burden on the caller to check the result
 - `std::string::npos`, `std::expected`
 - `std::optional` can be checked using `has_value()` or boolean conversion
- 4) Provide an **error status** as a side-effect
 - Requires reference parameter or global variable (bad)
 - Example: `std::istream`'s state (`good()`, `fail()`)

5) Throw an **exception****5.2.1. Exceptions**

throw value;

Any (copyable) type can be thrown and there are no means to specify what could be thrown. No meta-information is available as part of the exception. Exception thrown while exception is propagated results in program abort.

Example 10:

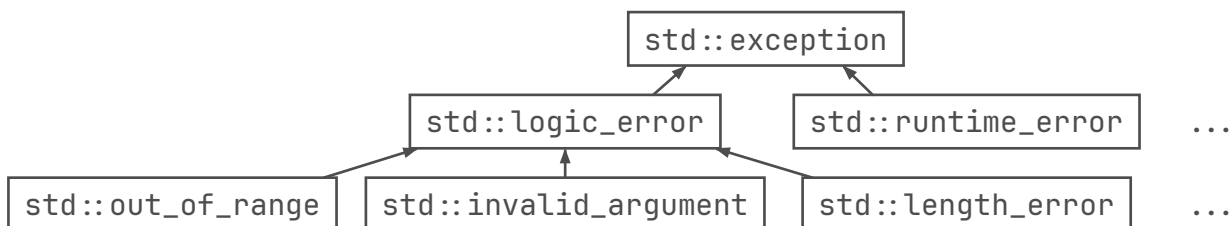
```
throw std::invalid_argument{"reason"};
throw 15;
```

Catching can be done using a try catch block, where the first match wins. Throw by value, catch by const reference avoids unnecessary copying and allows dynamic polymorphism for class types.

Example 11:

```
try {
    throwingCall();
} catch (type const & e) {
    //Handle type exception
} catch (type2 const & e) {
    //Handle type2 exception
} catch (...) {
    //Handle other exception types
}
```

The Standard Library has some pre-defined exception types that you can also use in `<stdexcept>`.



`std::exception` is the base class and provides the `what()` member function to obtain the “reason”, which is passed as a construction parameter.

Testing with Catch2 can be done as follows:

Example 12: REQUIRE_THROWS(code) when an exception is expected

```
TEST_CASE("square_root of negative value throws") {
    REQUIRE_THROWS(square_root(-1.0));
}
```

Example 13: `REQUIRE_THROWS_AS`(code, exception_type) when a specific type is expected to be thrown

```
TEST("at on empty vector throws std::out_of_range") {  
    std::vector<int> empty_vector{};  
    REQUIRE_THROWS_AS(empty_vector.at(0), std::out_of_range);  
}
```

Example 14: `REQUIRE_THROWS_WITH`(code, string or string-matcher) for exception content

```
TEST("parseInt of \"one\" throws with message") {  
    REQUIRE_THROWS_WITH(parseInt("one"), "parse error - invalid digits in  
'one'");  
}
```

You can make your program terminate when an exception is thrown by using the `noexcept` keyword after a function param definition.

```
auto add(int lhs, int rhs) noexcept → int {  
    return lhs + rhs;  
}
```

TODO:

lifetime extension through const & can only be done for temporary lifetimes?

BIBLIOGRAPHY

- [1] "C++ Mascot." Accessed: Sep. 12, 2026. [Online]. Available: https://media.licdn.com/dms/image/v2/D4D22AQFwZZ5wmpZmnw/feedshare-shrink_800/feedshare-shrink_800/0/1724256362408?e=2147483647&v=beta&t=OXxfC_GlaJFqlUgO03e-9B59Zpzu4-LYG90IM8k4UFE